GEN AI WITH LLMS · TOPIC 3

Training & Customising LLMs

Pre-training, Fine-tuning, and RL Post-training

Laura Wynter, PhD
SMU School of Computing and Information Sciences
IS469 · AY2026–27

ROADMAP

What we'll cover

01 Why customise, and why less, lately

02 Pre-training an LLM at scale

03 Training libraries & objectives

04 The customisation spectrum

05 Inside a training step: forward & backward

06 Cross-entropy loss, worked

07 Making training actually work

08 RL post-training: RLHF, RLVR & preference methods

09 Hands-on lab & what's next

FRAMING

Where we are

- T1: text processing, tokenisation, TF-IDF and word2vec embeddings
- T2: the transformer, attention (Q, K, V), encoders (BERT), the output head
- Today (T3): how the weights get made, pre-training, fine-tuning, RL post-training
- The through-line: one machinery, data in batches, a loss, gradient descent
- We build from a single training step up to how frontier models are aligned
- Next (T4): efficient fine-tuning, built directly on today's training loop

FRAMING

Why customise an LLM?

- Task-specific objective, e.g. medical or legal text; encoders (BERT) are almost always fine-tuned
- Improved performance on a target task, for example, better question answering
- Reducing bias in model outputs
- Adding recent knowledge the base model never saw
- Reality check: we do less full LLM fine-tuning these days, for reasons we'll meet later in the course

FRAMING

Learning objectives

- Distinguish pre-training vs. extended pre-training vs. fine-tuning
- Understand loss functions, and why cross-entropy
- Use the main training library: Hugging Face Transformers
- Understand the forward and backward passes of training
- Understand and be able to tune the key hyper-parameters
- Get a high-level grasp of RL post-training of an LLM

01

Pre-training an LLM

Tens of trillions of tokens, no labels required, where an LLM learns language, facts and skills.

What pre-training teaches

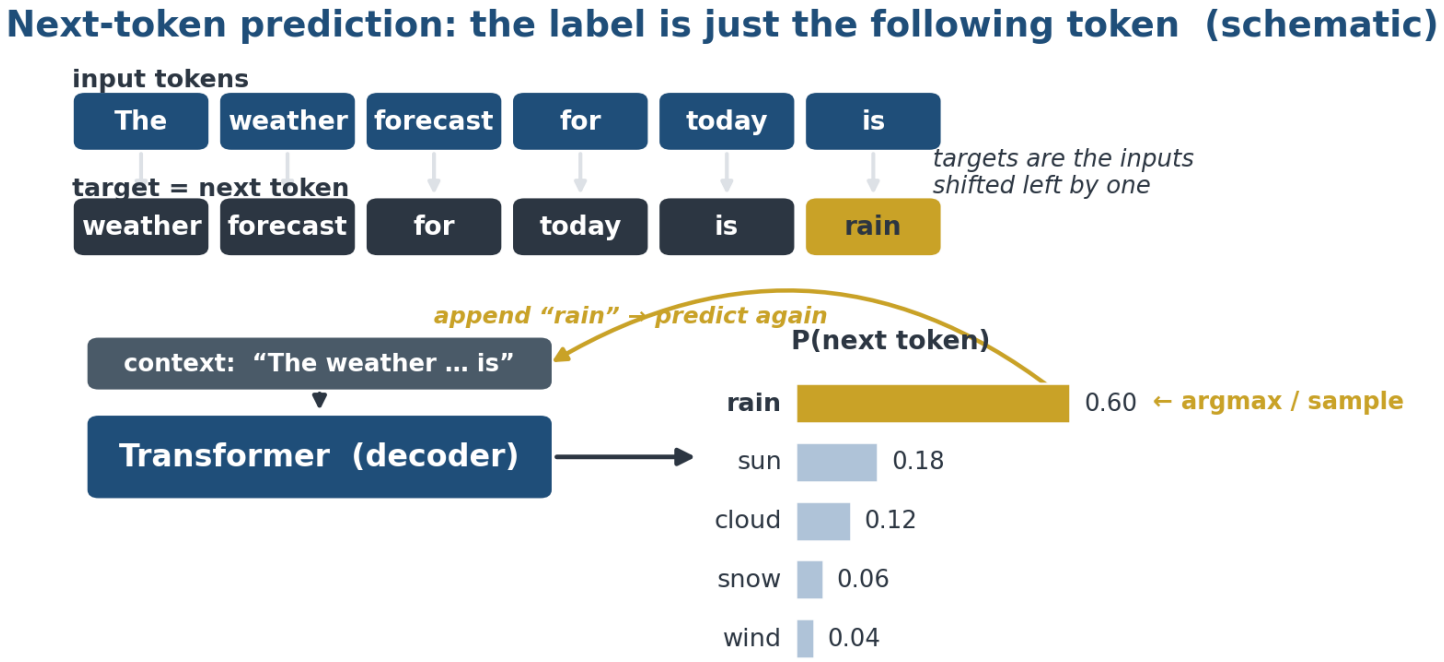
pump text into model to learn weights (majority of weights in multi attention & feed forward layer), these weights are given value during pre training

- Pre-training feeds the model tens of trillions of tokens
- (1) Language: grammar, sentence structure, word meaning, and contextual relationships
- (2) Facts & skills: e.g. how to write a formal letter vs. an email vs. a social post
- Pre-training a transformer is self-supervised, the data need not be labelled
- The 'label' is part of the input itself, masked during training (e.g. the next word)

pump massive amount of data into model to predict the next word

objective of training transformer model is to predict the next word

Next-token prediction, in one picture

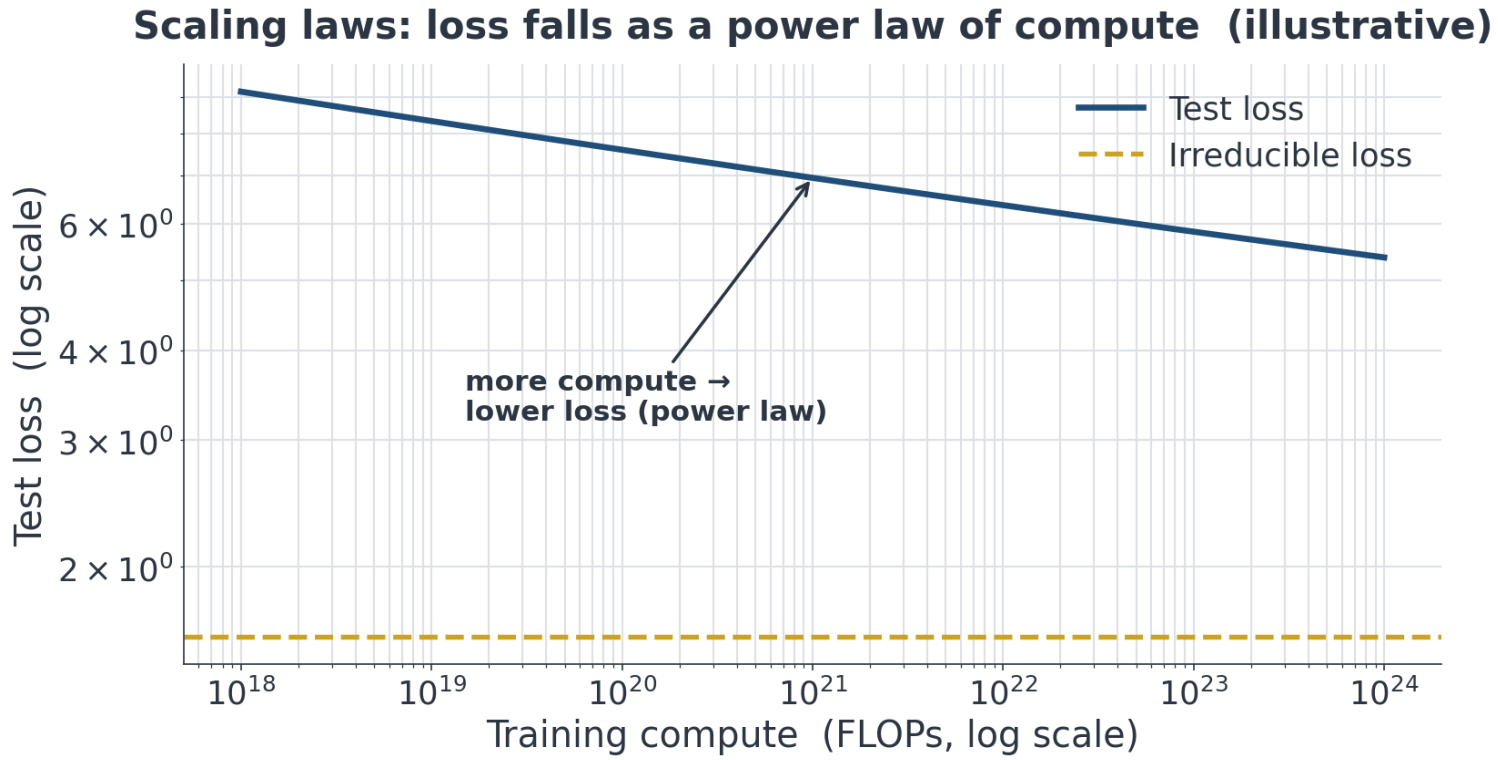


The target is simply each input shifted by one, the model learns a distribution over the vocabulary and the correct next token is its own 'label'.

Scale, data & self-supervision

- Trillions of tokens, and climbing: Llama 3 (2024) ~15T → Llama 4 (2025) >30T
- Chinchilla: ~20 tokens per parameter minimises the TRAINING RUN cost only
- Training is paid once, inference on every query, and tracks model size
- So labs over-train: a SMALLER model on far MORE tokens (8B ≈ 90× Chinchilla)
- You are unlikely to pre-train a decoder LLM from scratch
- You might pre-train an encoder for specialised, non-language tasks
- 'Customising' = further training a pre-trained model, i.e. fine-tuning

Why scale: loss follows a power law



More compute (and data, and parameters) drives test loss down a straight line on log-log, until it nears an irreducible floor.

How pre-training data becomes a model



Tens of trillions of tokens (Llama 4: >30T) with no labels, the 'answer' is just the next token itself.

Pre-training objective: MLM vs. CLM

Masked LM (MLM), encoders

- Used by BERT-style encoder models
- Mask tokens in the input, predict them
- Model sees both left and right context
- Great for understanding / classification

Causal LM (CLM), decoders

model only see words on the left

- Used by decoder (generative) models
- Predict the next token, left-to-right
- "The weather forecast for today is" → "rain."
- The objective behind text generation

02

The Customisation Spectrum

Same pre-trained weights, different objectives: extended pre-training, SFT, and instruction tuning.

Extended pre-training vs. supervised fine-tuning

Extended pre-training

- Keep the self-supervised causal loss
- Continue on domain-specific text
- Adds new knowledge / skills to the model
- No labels needed, same objective as pre-train

Supervised fine-tuning (SFT)

- Loss now uses labels (supervised)
- Each example has a correct answer
- Teaches format, style, task behaviour
- The basis for instruction & chatbot tuning

Four training regimes at a glance

Regime	Objective	Labels?	Purpose
Pre-training	Next-token (CLM / MLM)	No, self-sup.	Language, facts, skills
Extended pre-training	Same causal loss	No, self-sup.	Add domain knowledge
Supervised fine-tuning	Supervised loss	Yes	Task, format, behaviour
Instruction tuning	Supervised loss	Yes, Q&A pairs	Follow instructions, chat

Same machinery throughout, only the objective and the data change.

What SFT & instruction tuning are for

- Instruction tuning: teach the model to answer instructions in the right format and style (Q&A)
- Chatbot training: shape helpful, conversational responses
- Test-taking: math / science questions where each has a single correct answer
- Encoder (BERT-style) classification: every input maps to a correct label, exactly our lab task

03

Inside a Training Step

Start from the network itself: what training actually changes, and how one step changes it.

What are we actually training?

when training, we are learning value of the weights/parameters(w,b)

what is x in a large NN (transformer) : embedding of input

x is matrix where each row rep a token

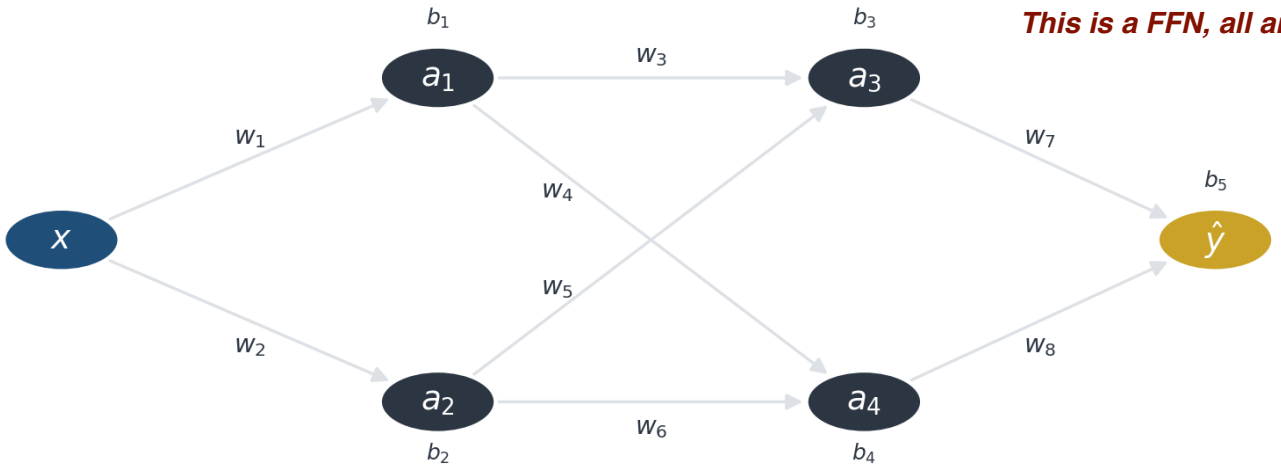
each token id associated with an starting input embedding

The network: weights and biases are what training changes

every w and every b is a number the model has to learn

training = adjust them until $\hat{y}$ matches the target y

This is a FFN, all arrows going in same direction



at the start the weights are random, so $\hat{y}$ is just a guess

A network is just weights w and biases b between layers of units. Training means adjusting every one of those numbers until the output $\hat{y}$ matches the target y. An LLM is this same picture with billions of weights.

Getting the data ready for training

passing all documents is computationally expensive —> must do so in batch

- Tokenise the dataset into IDs, then format it as PyTorch ("torch") tensors
- A dataloader shuffles the data and cuts it into batches of a chosen batch size
- Each batch drives exactly one update, that is one training step
- One full pass over the dataset is an epoch; 1M tokens ÷ 4096 ≈ 245 steps per epoch
- Then the same three things happen for every batch: forward pass, loss, backward pass

The forward pass

transformer for GenAI
- what kind of data used for training? Any data with text on it
-Forward pass by sending the data through neural network

If AI transformation model: what is an example of the target here? The target is the next word

initial values for w1 and w2: random numbers (1 or 0 etc.)

In GenAI model - the number of nodes in output is as many as the vocabluary

everytime u use a LLM, u are running a forward pass

- Send each batch through the network, left to right, a forward pass

when never be train before w and b are just random

- Initially, weights and biases are random values
- At each node: matmuls (matrix multiplications) with the current weights and biases, then an activation (ReLU, SwiGLU)
- Produce an output ŷ, then compute the loss L against the known target y
- The first forward pass, with random weights, is just guessing, the loss will be high

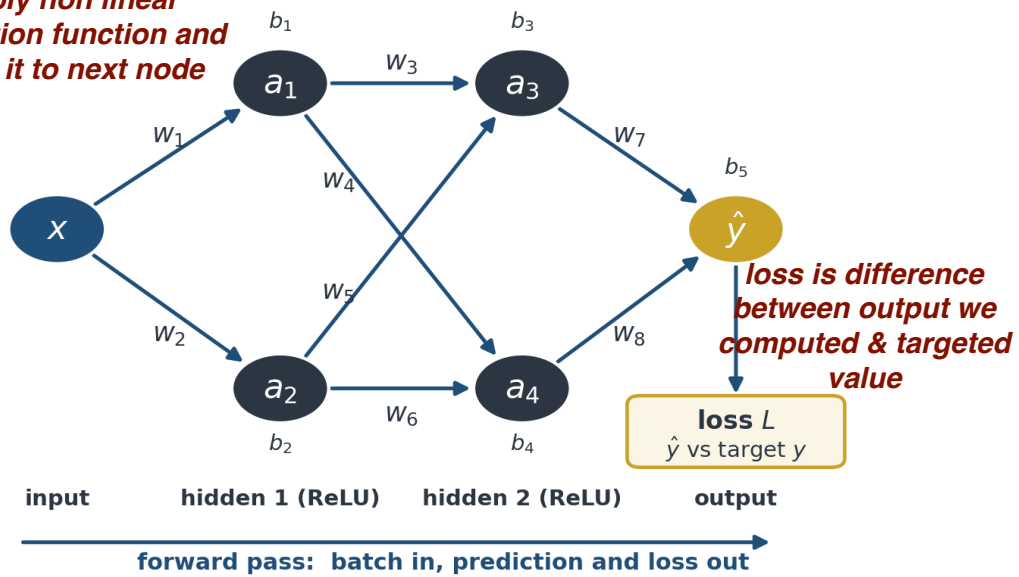
very first time loss will be quite high as initial weights are randomnised

Forward: signal in, a prediction and a loss out

$a_3 = \text{ReLU}(w_3 a_1 + w_5 a_2 + b_3)$ **at every node**

$\hat{y} = w_7 a_3 + w_8 a_4 + b_5$, then $L = (y - \hat{y})^2$

apply non linear activation function and send it to next node



loss is difference between output we computed & targeted value

Signal flows in at x, through the matmuls and activations, out to a prediction ŷ and a loss L.

finding how much each of this weight is responsible for the loss then adjust weights : Gradient Descent.

IF LLM and training on text, the output is just the next text. We always know the target output (y). So we can compute the loss.

loss is like error telling the weight (' youre responsible for this amount of error') - we know through gradient

The backward pass & gradient descent

purpose of backward pass is adjusting weights, next time we do forward pass the loss will be smaller. Which of the weight was responsible for that loss - adjust the weight. All weights responsible but to different degree

- We update weights and biases iteratively to reduce the loss

(-)because we want to descent to the minimum but the actual function slope is inc

how big we gonna take ? gama the step size

- Gradient descent: $w(n+1) = w(n) - \gamma \nabla L(n)$, γ is the learning rate

decreasing weight as function of gradient (but it doesnt mean the number is always getting smaller)

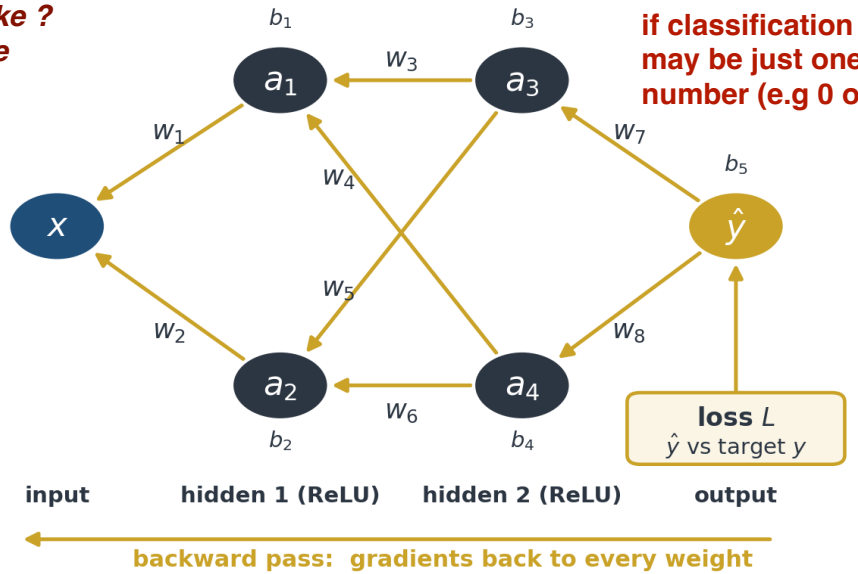
- Work backward from the output node toward the inputs, the same network, the other direction
- Each gradient says how much a weight nudged the loss; step every weight against its gradient
- Then repeat: forward → loss → backward → update, batch after batch

gradient descendant function: we trying to minimise this loss function

Backward: gradients from the loss back to the weights

$\partial L/\partial w$ for every weight, by the chain rule

then update each one: $w \leftarrow w - \gamma\, \partial L/\partial w$



if classification output may be just one output number (e.g 0 or 1)

Gradients ∂L/∂w flow from the loss back through the same network to every weight and bias.

Recap: the whole step as one loop

Training a transformer means running a forward pass on the transformer but this is just using

for a LLM it is a ouput vector of probability across vocab (FORWARD PASS), a different NN

One training step, as a loop: signal forward, gradients back (schematic)

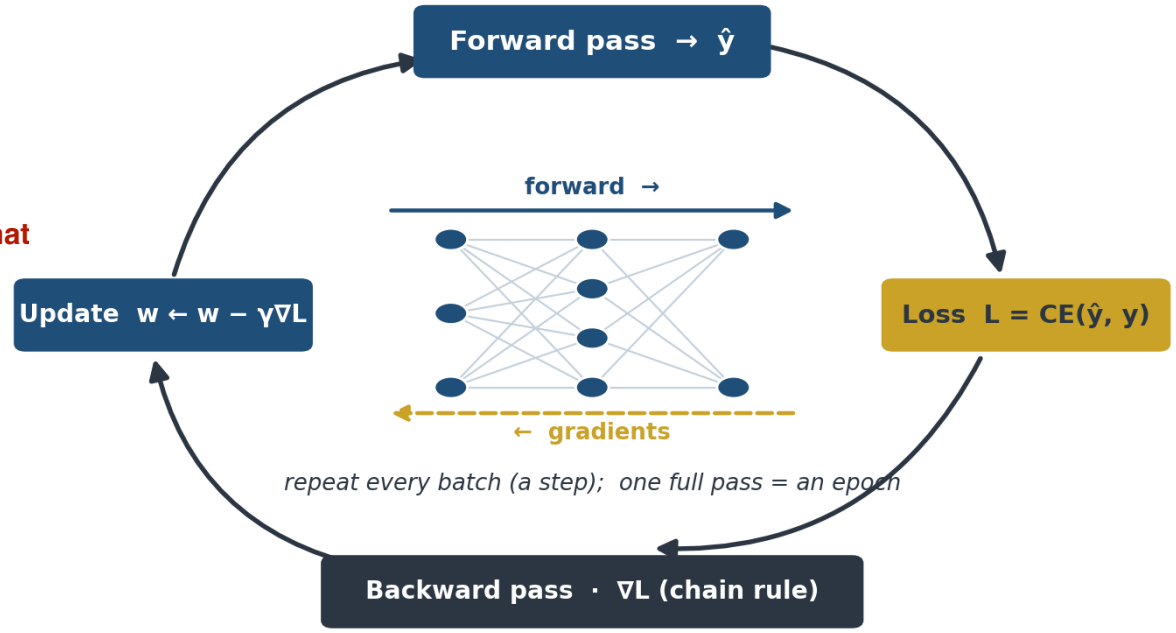
IN A TRANSFORMER (fwd pass):

what is nice about singapore?

Input x is what is nice about singapore the output is the token the now need another forward pass (now input is what is nice about singapore the) now next token may be food

what is a target output for the LLM,actual output to be produce

input is all of text we want to train model on. we hide next word but we know the next word because is in the text



Everything we just walked through, in one picture: forward to a prediction and a loss, gradients back to every weight, update, repeat every batch.

04

Cross-Entropy Loss

The loss LLMs actually train on: negative log-likelihood of the true next token.

negative log likelihood of the next token

How to compute the loss: cross-entropy

we don't use y- y^ actually.we dont use MSE in genAI

- Usually cross-entropy (CE) loss is used rather than MSE loss
- CE loss is the negative log-likelihood of the target token j (the true next token)
- CE loss is smooth and differentiable, hence well-suited to gradient-based optimisation
- The log in the CE loss is sensitive to changes in high-confidence predictions, so the model stays cautious when uncertain
- The more confident the model is on the correct outcome, the lower the loss; more confident on a wrong outcome, the higher the loss

small k here is the particular word in vector

Cross-entropy loss, example 1

K is how many tokens we have in vocab

y is the ground truth here - a vector with single 1 and all 0s

probability of k token being the next word

- $L = -\Sigma_{k=1..K}\, y_k \cdot \log(p_k)$
- When $y_k = 1$, the term $-\log(p_k)$ decreases (gets better) as the predicted probability p_k increases
- A wrong class has $y_k = 0$ so it does not enter the loss directly, but because $\Sigma_i\, p_i = 1$, raising a wrong class's probability forces the correct one down
- The log is more sensitive to changes in high than low values: raising p from 0.98 to 0.99 lowers the loss less than raising it from 0.51 to 0.52
- So an "I don't know" prediction (all probabilities equal) gives a lower loss than a very confident but wrong one

· Source: Chris Hughes, "A brief overview of cross-entropy loss"

Cross-entropy loss, example 2

- 4-token vocabulary [**the**, **man**, **eats**, **fish**]; true next token is **man**, so the label is [0, 1, 0, 0]
- Only the true class contributes: $L = -\Sigma_k y_k \cdot \log(p_k) = -\log(p_{man})$
- Probabilities below are in that order: **the**, **man**, **eats**, **fish**
- pred [0.01, **0.60**, 0.09, 0.30] (60% on **man**): $L = -\log 0.60 = 0.22$
- more confident [0.01, **0.80**, 0.09, 0.10]: $L = 0.097$ (lower is better)
- wrong & confident [0.01, **0.15**, 0.09, 0.84]: $L = 0.82$ (worst)
- "doesn't know" [0.25, **0.25**, 0.25, 0.25]: $L = -\log 0.25 = 0.60$ (still better)

y2=1 and all of the other ys are 0

probability is higher on the correct ouput then the loss is lower

more confident means higher probability

model high confident 0.84 that is fish but its wrong so the lost is very high. Loss function penalises model when is confident but wrong

higher loss (worst)

equal probability == idk

better to say idk than wrong & confident

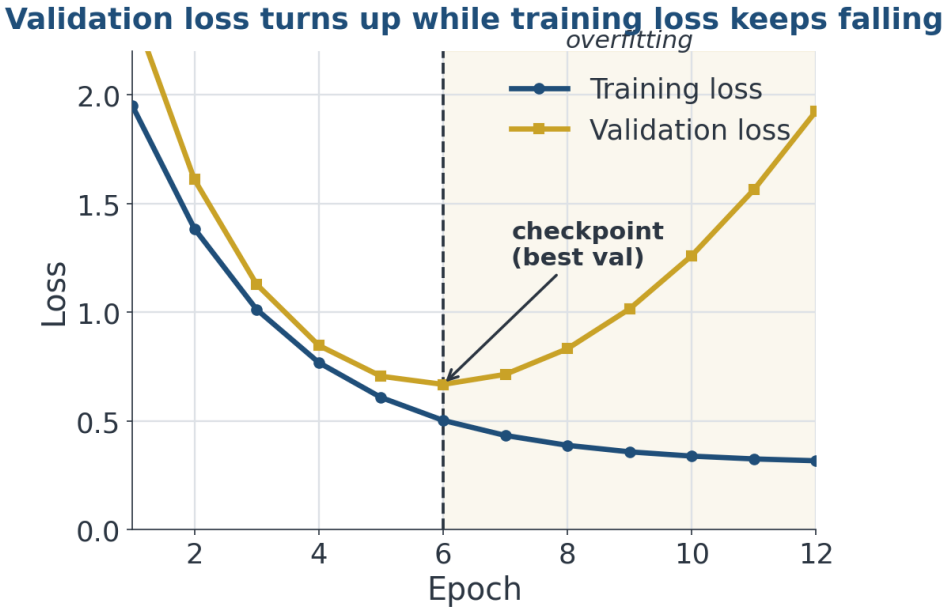
05

Making Training Actually Work

Overfitting, dropout, the AdamW optimiser, batches, and the hyper-parameters you'll tune.

Watch validation loss, not training loss

- Train over many epochs and the LLM starts to memorise the training data
- Training loss keeps trending down, that alone tells you nothing about generalisation
- When validation loss (on held-out data) turns back up, you're overfitting
- Pick the checkpoint from just before that turn, the dashed line
- This train-vs-val gap is the single most important thing to watch during fine-tuning



Validation loss bottoms out, then rises as the model overfits; checkpoint at the minimum.

Dropout: regularising to avoid overfitting

reduce dimension of training (cross out some weights) to make function less perfect when it comes to estimating the data which becomes a better estimator for the data we don't have

- Billion-parameter transformers overfit easily, worse with small or domain-specific datasets
- Dropout randomly zeroes a percentage of a layer's outputs during training
- Applied to both attention and feed-forward sub-layers
- In attention, some elements are dropped from the softmax term
- Forces the network not to rely on any single unit, only active at training time

Batches, OOM & gradient accumulation

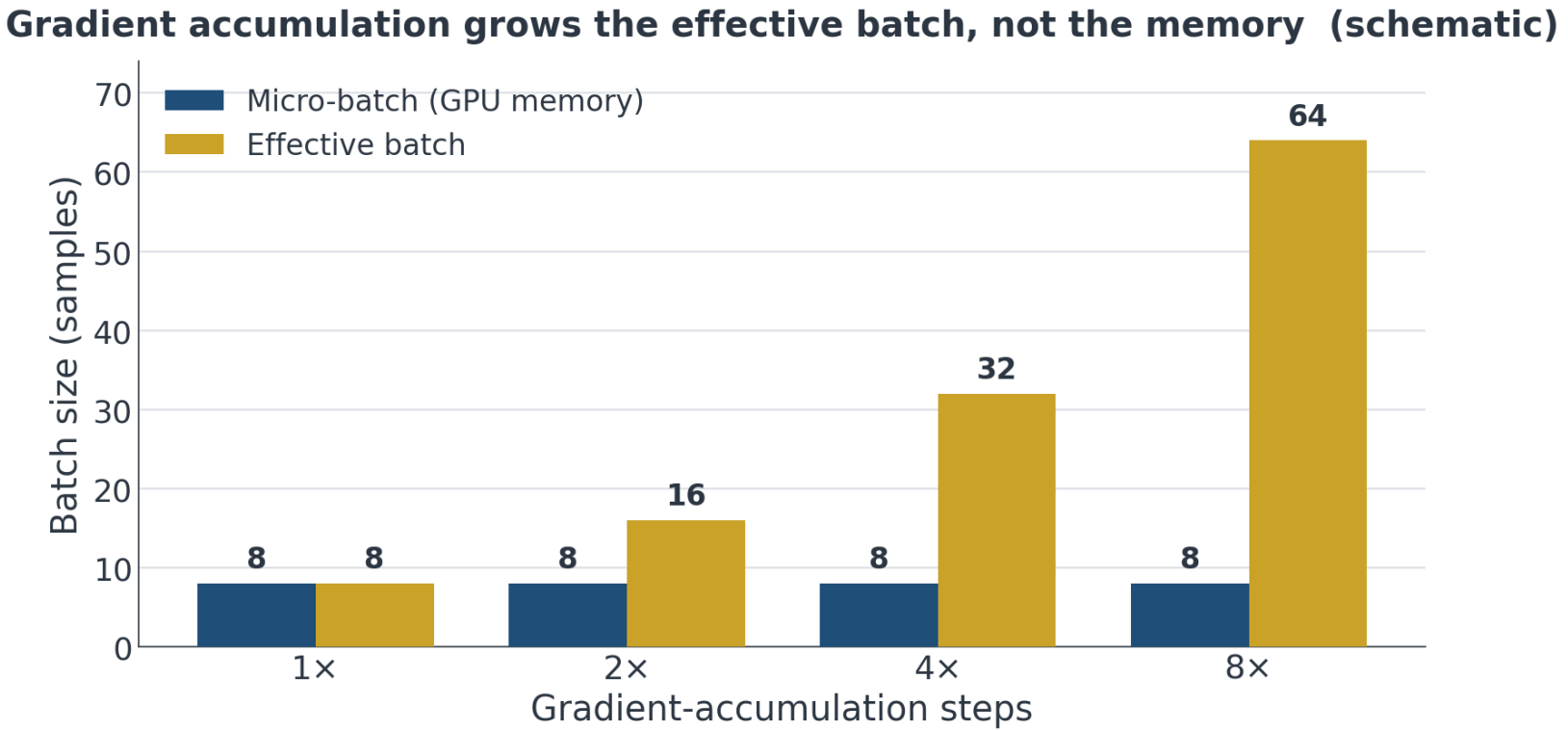
everytime we send through one page, dont update weights yet because the model hasnt seen other pages.

- Batch size shapes memory use, learning dynamics and throughput, for both, we want large batches

gathering info from each batch fwd pass, then create single backward pass from the info it accumulated

- But large batches overflow GPU memory (OOM), capping how big you can go
- Gradient accumulation: split a big batch into subsets, accumulate gradients, update once at the end
- Find the limit empirically: start at batch size 1, double, remeasure, until throughput drops or OOM
- Judge learning quality by validation loss on held-out data, not training loss

Gradient accumulation, visualised



Accumulate gradients over several micro-batches, then update once, a big effective batch at a small batch's memory cost.

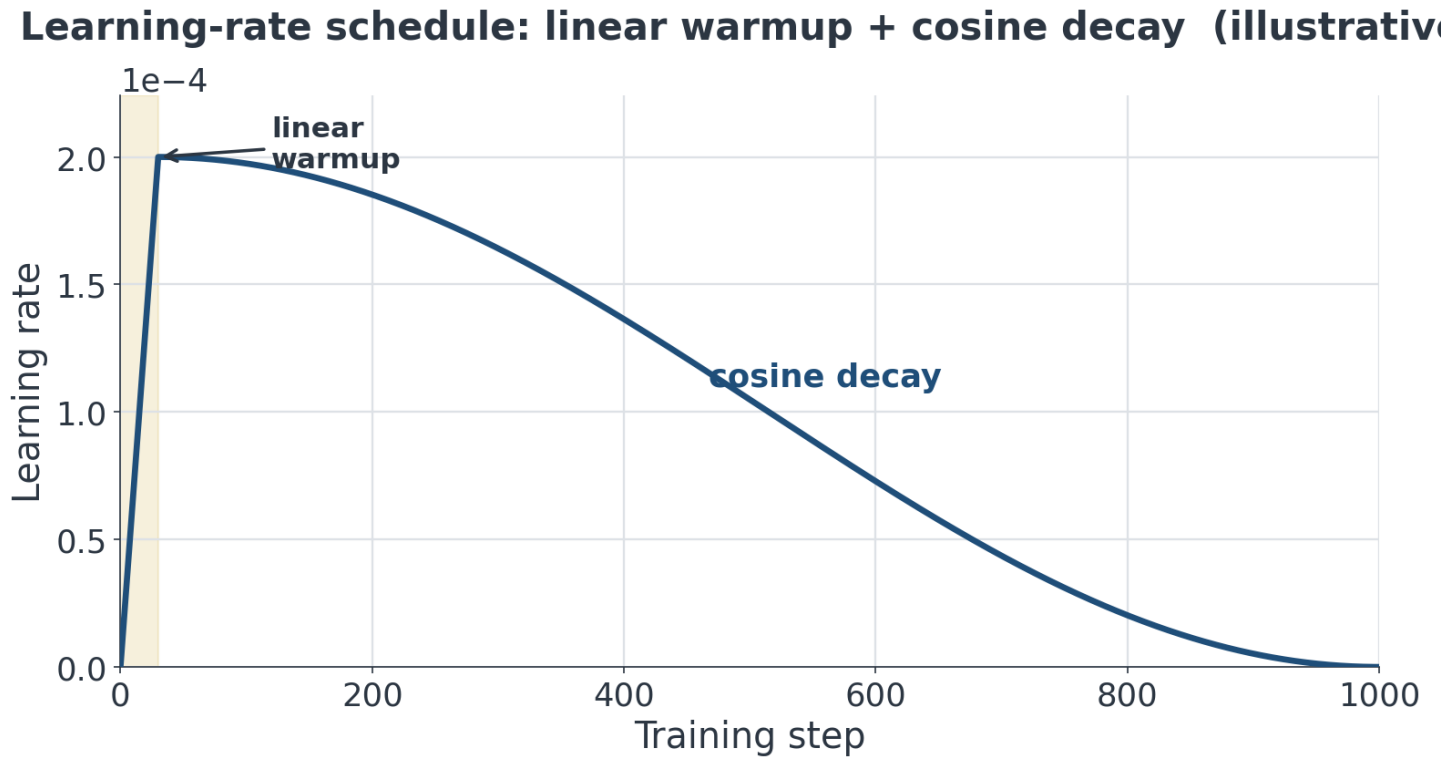
different aspect of training to adjust

The hyper-parameters you'll tune

Hyper-parameter	What it controls	Typical setting
Learning rate + schedule	Step size over training	linear / cosine decay
Warmup ratio	LR ramp-up at the start	0.03, first 3% of steps
Batch size	Samples per update	as large as memory allows
Grad-accum steps	Simulate a bigger batch	2 – 16
Number of epochs	Passes over the data	few, watch val loss

A hyper-parameter is set before training; judge each choice by validation loss, not training loss.

A learning-rate schedule: warmup then decay



Ramp the learning rate up over the first few percent of steps, then cosine-decay it toward zero, the default recipe.

06

RL Post-Training

After SFT: reinforcement learning to align the model with human preferences and correctness.

RL from Human Feedback (RLHF)

In transformer training:
each transformer model to predict next token is like learning the model language, facts and scales , sentence strucutres, by predicting next token is predicting answer to qns

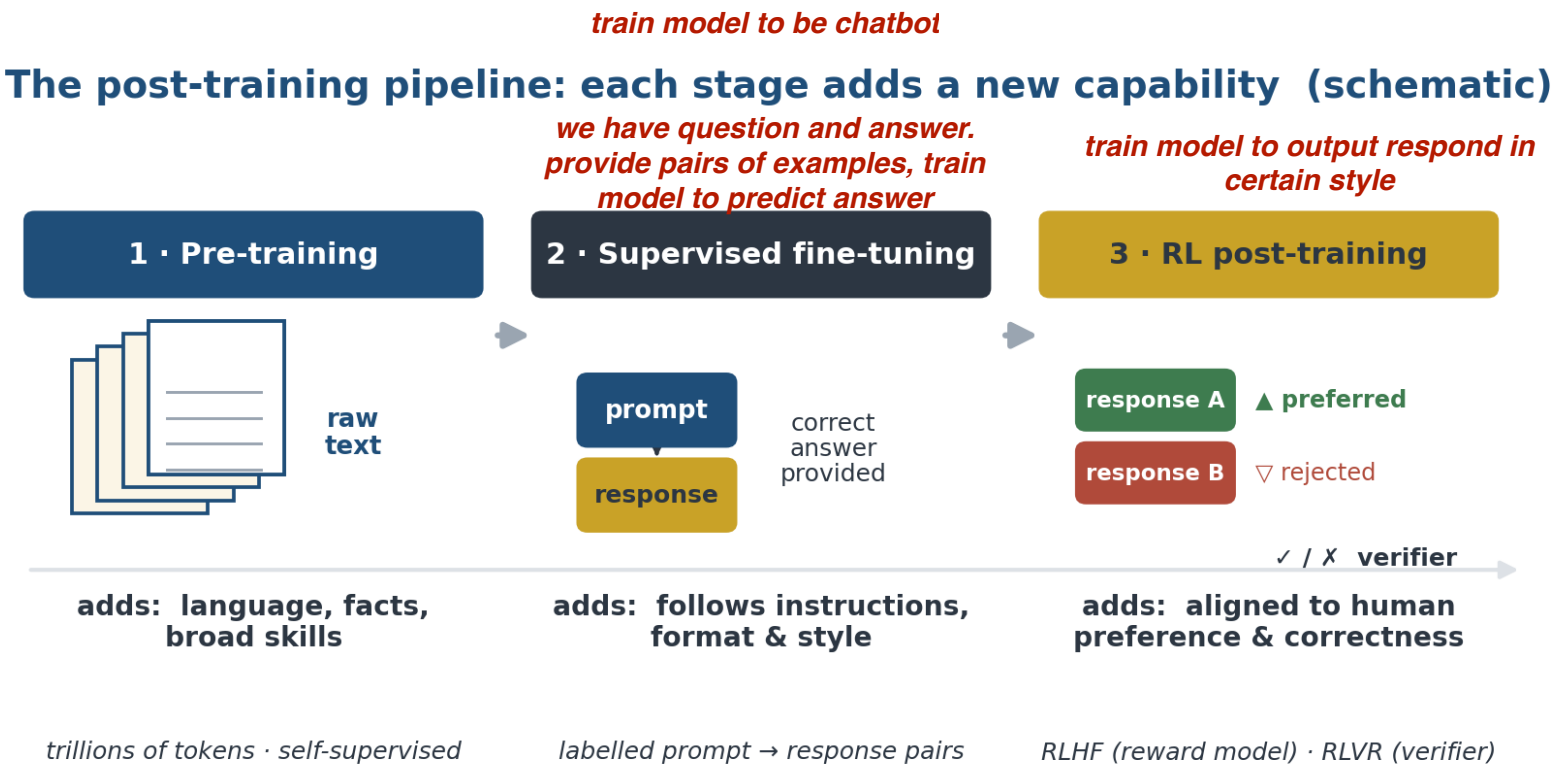
- A 3-step process for aligning LLMs to human preferences
- (1) Start from a model already post-trained with SFT
- (2) Reward model: humans rank multiple responses per prompt; train a separate NN to predict those preferences
- (3) RL phase: adjust the LLM's weights to maximise the reward the reward model predicts, then repeat
- Human feedback is distilled once into a reward model, which then guides fine-tuning at scale

Where RL fits: the post-training pipeline

only req gathering documents & batching them into size to fit into memory

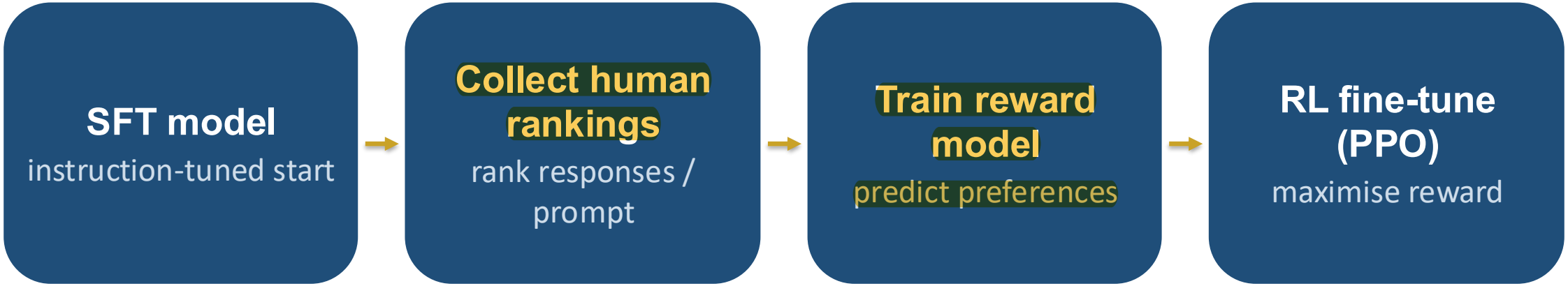
input: is all the text of the internet
pre training objective: predict the next word

all ground truth already there we dont have to label because it is just the next word



Pre-training builds broad ability, SFT teaches instruction-following, and RL post-training (RLHF / RLVR) aligns the model to preference and correctness.

RLHF in four steps



Human feedback is distilled once into a reward model, which then guides fine-tuning at scale. ⟳ iterate.

RL from Verifiable Rewards (RLVR)

- Used in DeepSeek post-training, like RLHF but with a binary reward: 1 = correct, 0 = incorrect
- A programmatic verifier sets the reward, no separate neural reward model to train
- Ideal for math and coding, where correctness can be checked automatically
- Won't work where outputs can't be programmatically verified
- The LLM's output is checked by the verifier; weights are updated toward correct outputs

Reasoning models & test-time compute

- RL on verifiable rewards (RLVR / GRPO) trains models to emit a long chain-of-thought and SELF-CORRECT mid-answer
- Accuracy then scales with inference-time tokens, spend more 'thinking' tokens, get better answers
- This 'test-time compute' is a new axis of scaling: not just a bigger model, but longer reasoning at inference
- The products: DeepSeek-R1 (Jan 2025) and o-series-style reasoning models
- This is the payoff of the RLVR / GRPO slides above, what that training actually buys you

Key takeaways

01

It's all one loop

Batches → forward pass → loss → backward pass → update. Pre-training and fine-tuning share the same machinery.

02

Cross-entropy is the loss

Logits → softmax → negative log-likelihood of the true token. Confidently wrong is punished hardest.

03

Watch validation, not training loss

Overfitting is the enemy: dropout, weight decay, and checkpoint on the val curve.

04

Alignment is RL post-training

RLHF learns a reward model; RLVR verifies correctness; DPO / GRPO make preference training lighter.

LAB

Hands-on lab: fine-tune & log the curves

- Fine-tune google-bert/bert-base-cased as a sequence classifier on a small subset of yelp_review_full (~200 train / 100 val / 100 test) so it runs fast in class
- Use the Hugging Face Trainer: tokenise, set TrainingArguments, train, evaluate
- Log training and validation loss every epoch so you can plot both curves
- Run 2, 3 configurations (learning rate, batch size, epochs) and overlay their loss curves
- Use gradient accumulation to simulate a larger effective batch within Colab's memory

LAB

Hands-on lab: read the training curves

- Find where validation loss bottoms out and then turns back up, that is the onset of overfitting
- Select the checkpoint at the validation-loss minimum, not the final epoch
- Compare configurations: which learning rate and schedule generalise best, and why?
- Connect it back to the theory: dropout, weight decay and early checkpointing all move that curve
- This fine-tune → evaluate loop is the foundation we formalise into proper evaluation in Topic 6